Министерство науки и высшего образования Российской Федерации
федеральное государственное автономное образовательное учреждение
высшего образования
«Санкт-Петербургский политехнический университет Петра Великого»
(ФГАОУ ВО СПбПУ)
Институт компьютерных наук и кибербезопасности
Высшая школа компьютерных технологий и информационных систем

ПРАКТИЧЕСКАЯ РАБОТА

По дисциплине «Объектно-ориентированное программирование»

Прогулка по Лондону

Студент гр. 5140901/51501 Н.С.Пиюренко

подпись, дата инициалы и фамилия

Оценка выполненной студентом работы:

Руководитель,
д.т.н, профессор ВШКТиИС. Г.Ф.Малыхина

подпись, дата инициалы и фамилия

Санкт-Петербург – 2025

САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА ВЕЛИКОГО

Институт компьютерных наук и кибербезопасности

Высшая школа компьютерных технологий и информационных систем

ЗАДАНИЕ

на выполнение практической работы по дисциплине

«Объектно-ориентированное программирование»

студенту Пиюренко Никите Сергеевичу группа: 5140901/51501

1. Тема работы: Создание программы на языке программирования Java, прогулка по лондонскому метро.

2. Срок сдачи студентом законченной работы: 05.12.2025

СОДЕРЖАНИЕ

1 Общие сведения 4
1.1 Идея программы 4
1.2 Основные правила 4
2 Реализация классов 6
2.1 Класс LondonTubeMap 6
2.2 Класс Station и Connection 6
2.3 Инициализация транспортной сети 8
3 Алгоритм навигации и поиска пути 10
3.1 Реализация алгоритма Дейкстры 10
3.2 Верояиностная модель принятия решений 10
3.3 Обновление Utility 13
4 Реализация графического интерфейса 14
4.1 Класс MapPanel 14
4.2 Визуализация элементов карты 15
4.3 Отображение агента и служебной информации 16
5 Система управления агентом 17
5.1 Инициализация и запуск агента 17
5.2 Пошаговое выполнение и принятие решений 17
5.3 Управление и сброс состояния 18
Приложение 19

1 Общие сведения

1.1 Идея программы

Данный программный продукт представляет собой интерактивную имитационную модель навигации интеллектуального агента в системе лондонского метрополитена, разработанную с использованием языка программирования Java и библиотеки Swing для создания графического интерфейса пользователя. Основная концепция программы заключается в демонстрации работы алгоритмов поиска пути и методов обучения с подкреплением в контексте транспортной сети.

Программа реализует два основных подхода к навигации: детерминированный алгоритм Дейкстры для нахождения кратчайшего пути и вероятностную модель на основе utility-функций для имитации интеллектуального поведения агента. Алгоритм Дейкстры обеспечивает нахождение оптимального маршрута по критерию минимального расстояния, в то время как utility-функции позволяют агенту принимать решения в условиях неопределенности и изменяющихся условий среды.

Визуальное представление карты метро с цветовым кодированием линий, отображением расстояний и utility-значений создает комплексную среду для наблюдения за процессами принятия решений. Интерфейс позволяет пользователю взаимодействовать с картой, выбирать станции отправления и назначения, а также контролировать поведение агента в реальном времени

1.2 Основные правила

Для обеспечения корректной работы навигационной системы были реализованы следующие фундаментальные правила и структуры данных. Игровое пространство представляет собой упрощенную схему лондонского

метрополитена, содержащую четыре линии: красную, желтую, зеленую и синюю, с общей сложностью 15 станций.

Каждая станция характеризуется координатами на плоскости, названием и принадлежностью к одной или нескольким линиям метро. Соединения между станциями представлены в виде взвешенного графа, где веса ребер соответствуют расстояниям между станциями в километрах. Особое внимание уделено станциям пересадок, которые позволяют агенту перемещаться между различными линиями метро.

Система навигации агента основана на марковской модели принятия решений, где каждое состояние соответствует станции метро, а действия - возможным перемещениям к соседним станциям. Функция вознаграждения стимулирует агента к достижению целевой станции и penalizes за нахождение в удаленных от цели состояниях.

2 Реализация классов

2.1 Класс LondonTubeMap

Класс LondonTubeMap является центральным компонентом приложения, наследуется от JFrame и реализует весь графический интерфейс пользователя, а также основную логику моделирования и навигации. Архитектура класса организована вокруг управления состоянием карты метро, вычисления маршрутов и контроля поведения агента.

Основные поля класса включают константы, определяющие размеры окна отображения (1200x800 пикселей) и отступы для корректного позиционирования элементов. Коллекции stations и graph хранят информацию о станциях и соединениях между ними, образуя основу транспортной сети.

Система управления агентом реализована через набор переменных: agentCurrentStation отслеживает текущее положение агента, agentPath хранит историю перемещений, agentSteps считает количество выполненных шагов, а agentReachedTarget указывает на достижение цели. Вероятностная модель использует коэффициент дисконтирования gamma и карту utilities для хранения значений полезности станций.

2.2 Класс Station и Connection

Класс Station инкапсулирует свойства станции метро, включая название, координаты позиционирования на карте и список линий, к которым принадлежит станция. Метод addLine обеспечивает возможность добавления новых линий к существующей станции, что особенно важно для станций пересадок.

```java
class Station {  37 usages
    String name;  5 usages
    int x, y;  30 usages
    List<String> lines;  10 usages

    Station(String name, int x, int y, List<String> lines) {
        this.name = name;
        this.x = x;
        this.y = y;
        this.lines = lines;
    }

    void addLine(String line) {  no usages
        if (!lines.contains(line)) {
            lines.add(line);
        }
    }
}
```

Рисунок 2.2.1 — Реализация класса Station

Класс Connection представляет соединение между двумя станциями и содержит информацию о станциях отправления и назначения, расстоянии между ними и принадлежности к конкретной линии метро. Эта структура данных является фундаментальной для построения графа транспортной сети и вычисления маршрутов.

```java
class Connection {  17 usages
    String from, to;  9 usages
    double distance;  5 usages
    String line;  4 usages

    Connection(String from, String to, double distance, String line) {
        this.from = from;
        this.to = to;
        this.distance = distance;
        this.line = line;
    }
}
```

Рисунок 2.2.2 — Связь между двумя станциями

Система соединений поддерживает как обычные перемещения между разными станциями, так и специальные переходы "Transfer" для моделирования пересадок между линиями на одной станции. Это позволяет корректно обрабатывать сложные транспортные узлы.

2.3 Инициализация транспортной сети

Метод initializeStations создает и размещает станции на карте с учетом логической структуры метрополитена. Расположение станций организовано по уровням: верхняя линия (красная) размещена в верхней части карты, средние линии (желтая и зеленая) – в центральной части, а дополнительные станции (синяя линия) - в различных секторах для создания реалистичной схемы.

```
private void initializeStations() {  1 usage
    // Опускаем карту ниже - смещаем координаты Y вниз
    int centerX = WIDTH / 2;
    int centerY = HEIGHT / 2 + 100; // Смещаем центр ниже

    // Верхняя линия (красная)
    stations.put("Baker St.", new Station( name: "Baker St.",  x: centerX - 400,  y: centerY - 150, Arrays.asList("Red")));
    stations.put("King's Cross st.", new Station( name: "King's Cross st.",  x: centerX - 200,  y: centerY - 150, Arrays.asList("Red", "Green")));
    stations.put("Moorgate", new Station( name: "Moorgate", centerX,  y: centerY - 150, Arrays.asList("Red", "Blue")));
    stations.put("Liverpool st st.", new Station( name: "Liverpool st st.",  x: centerX + 200,  y: centerY - 150, Arrays.asList("Red", "Yellow")));
    stations.put("Tower Hill", new Station( name: "Tower Hill",  x: centerX + 400,  y: centerY - 150, Arrays.asList("Red")));

    // Средняя линия (желтая)
    stations.put("Bond st.", new Station( name: "Bond st.",  x: centerX - 400,  y: centerY - 50, Arrays.asList("Yellow")));
    stations.put("Oxford circ.", new Station( name: "Oxford circ.",  x: centerX - 200,  y: centerY - 50, Arrays.asList("Yellow")));
    stations.put("Chancery Lane", new Station( name: "Chancery Lane", centerX,  y: centerY - 50, Arrays.asList("Yellow", "Green")));
    stations.put("Bank", new Station( name: "Bank",  x: centerX + 200,  y: centerY - 50, Arrays.asList("Yellow", "Blue")));

    // Нижняя линия (зеленая)
    stations.put("Covent Garden", new Station( name: "Covent Garden",  x: centerX - 200,  y: centerY + 50, Arrays.asList("Green")));
    stations.put("Piccadilly circ", new Station( name: "Piccadilly circ",  x: centerX - 200,  y: centerY + 150, Arrays.asList("Green")));
    stations.put("Green Park", new Station( name: "Green Park",  x: centerX - 200,  y: centerY + 250, Arrays.asList("Green")));

    // Дополнительные станции (синяя)
    stations.put("Old st", new Station( name: "Old st", centerX,  y: centerY - 250, Arrays.asList("Blue")));
    stations.put("Monument", new Station( name: "Monument",  x: centerX + 200,  y: centerY + 50, Arrays.asList("Blue")));
}
```

Рисунок 2.3.1 — Инициализация станций метрополитена

Метод initializeConnections устанавливает соединения между станциями в соответствии с линиями метро. Каждое соединение характеризуется расстоянием и принадлежностью к конкретной линии. Особое внимание уделено созданию переходов между линиями через станции пересадок, что обеспечивает связность всей транспортной сети.

```java
private void initializeConnections() {  1 usage
    // Красная линия
    addConnection( from: "Baker St.",  to: "King's Cross st.",  distance: 2.1,  line: "Red");
    addConnection( from: "King's Cross st.",  to: "Moorgate",  distance: 1.8,  line: "Red");
    addConnection( from: "Moorgate",  to: "Liverpool st st.",  distance: 1.2,  line: "Red");
    addConnection( from: "Liverpool st st.",  to: "Tower Hill",  distance: 1.5,  line: "Red");

    // Желтая линия
    addConnection( from: "Bond st.",  to: "Oxford circ.",  distance: 1.3,  line: "Yellow");
    addConnection( from: "Oxford circ.",  to: "Chancery Lane",  distance: 1.5,  line: "Yellow");
    addConnection( from: "Chancery Lane",  to: "Bank",  distance: 1.2,  line: "Yellow");
    addConnection( from: "Bank",  to: "Liverpool st st.",  distance: 1.0,  line: "Yellow");

    // Зеленая линия
    addConnection( from: "King's Cross st.",  to: "Chancery Lane",  distance: 1.4,  line: "Green");
    addConnection( from: "Chancery Lane",  to: "Covent Garden",  distance: 1.6,  line: "Green");
    addConnection( from: "Covent Garden",  to: "Piccadilly circ",  distance: 0.8,  line: "Green");
    addConnection( from: "Piccadilly circ",  to: "Green Park",  distance: 1.1,  line: "Green");

    // Синяя линия
    addConnection( from: "Old st",  to: "Moorgate",  distance: 0.9,  line: "Blue");
    addConnection( from: "Moorgate",  to: "Bank",  distance: 1.3,  line: "Blue");
    addConnection( from: "Bank",  to: "Monument",  distance: 0.5,  line: "Blue");

    // Пересадки между линиями
    addConnection( from: "King's Cross st.",  to: "King's Cross st.",  distance: 0.0,  line: "Transfer");
    addConnection( from: "Chancery Lane",  to: "Chancery Lane",  distance: 0.0,  line: "Transfer");
    addConnection( from: "Bank",  to: "Bank",  distance: 0.0,  line: "Transfer");
    addConnection( from: "Moorgate",  to: "Moorgate",  distance: 0.0,  line: "Transfer");
    addConnection( from: "Liverpool st st.",  to: "Liverpool st st.",  distance: 0.0,  line: "Transfer");
}
```

Рисунок 2.3.2 — Соединение между станциями

Система автоматического построения графа в методе buildGraph использует информацию о станциях и соединениях для создания структуры данных, оптимальной для алгоритмов поиска пути. Граф представлен в виде словаря, где ключами являются названия станций, а значениями – списки исходящих соединений.

3 Алгоритм навигации и поиска пути

3.1 Реализация алгоритма Дейкстры

Метод Дейкстра реализует классический алгоритм поиска кратчайшего пути в взвешенном графе. Алгоритм использует приоритетную очередь для эффективного выбора следующей вершины для обработки и два вспомогательных словаря: distances для хранения текущих кратчайших расстояний и previous для восстановления пути.

На каждой итерации алгоритм извлекает из очереди станцию с минимальным известным расстоянием и обновляет расстояния до всех смежных станций. Если найден более короткий путь до соседней станции, соответствующая запись в словаре distances обновляется, а в словарь previous записывается информация о предыдущей станции на кратчайшем пути.

После завершения работы алгоритма метод reconstructs путь от целевой станции к начальной, используя информацию из словаря previous. Результирующий маршрут представляет собой последовательность станций, обеспечивающую минимальное суммарное расстояние между начальной и конечной точками.

3.2 Верояиностная модель принятия решений

Система utility-функций реализована для имитации интеллектуального поведения агента в условиях неопределенности. Метод getReward определяет функцию вознаграждения, которая присваивает высокое положительное вознаграждение за достижение целевой станции и отрицательные вознаграждения за нахождение в других станциях, пропорционально их удаленности от цели.

```java
private double getReward(String station, String target) {  1 usage
    if (station.equals(target)) {
        return 100.0; // Большое вознаграждение за достижение цели
    }

    // Отрицательное вознаграждение пропорционально расстоянию
    Station currentStation = stations.get(station);
    Station targetStation = stations.get(target);

    if (currentStation != null && targetStation != null) {
        double distance = Math.sqrt(
                Math.pow(currentStation.x - targetStation.x, 2) +
                        Math.pow(currentStation.y - targetStation.y, 2)
        );
        return -distance / 100.0;
    }

    return -1.0; // Стандартное отрицательное вознаграждение
}
```

Рисунок 3.2.1 — Функция вознаграждения

Метод getTransitionProbability моделирует вероятностный характер перемещений в транспортной системе. Вероятность перехода вычисляется на основе расстояния между станциями и направления движения относительно цели. Перемещения в сторону целевой станции получают повышенные веса, что отражает разумное поведение агента.

```java
private double getTransitionProbability(String from, String to, String target) {  1 usage
    List<Connection> connections = graph.get(from);
    if (connections == null) return 0.0;

    // Базовые вероятности на основе расстояния
    double totalWeight = 0.0;
    Map<String, Double> weights = new HashMap<>();

    for (Connection conn : connections) {
        if (!conn.from.equals(conn.to)) { // Исключаем пересадки на себя
            double weight = 1.0 / (conn.distance + 0.1); // Чем ближе, тем вероятнее

            // Увеличиваем вес для направлений к цели
            if (isMovingTowardTarget(conn.to, from, target)) {
                weight *= 2.0;
            }

            weights.put(conn.to, weight);
            totalWeight += weight;
        }
    }

    if (totalWeight == 0.0 || !weights.containsKey(to)) {
        return 0.0;
    }

    return weights.get(to) / totalWeight;
}
```

Рисунок 3.2.2 — Вероятность перехода

Функция calculateUtility реализует уравнение Беллмана для вычисления полезности состояний. Значение полезности станции рассчитывается как сумма

немедленного вознаграждения и дисконтированной ожидаемой полезности следующего состояния. Этот подход обеспечивает учет долгосрочных последствий принимаемых решений.

```
private double calculateUtility(String station, String target) {  1 usage
    double reward = getReward(station, target);

    if (station.equals(target)) {
        return reward; // Целевое состояние
    }

    List<Connection> connections = graph.get(station);
    if (connections == null) return reward;

    double maxExpectedUtility = Double.NEGATIVE_INFINITY;

    // Для каждого возможного действия (перехода) вычисляем ожидаемую utility
    for (Connection conn : connections) {
        if (!conn.from.equals(conn.to)) {
            String nextStation = conn.to;
            double transitionProb = getTransitionProbability(station, nextStation, target);
            double expectedUtility = transitionProb * utilities.getOrDefault(nextStation,  defaultValue: 0.0);

            if (expectedUtility > maxExpectedUtility) {
                maxExpectedUtility = expectedUtility;
            }
        }
    }

    if (maxExpectedUtility == Double.NEGATIVE_INFINITY) {
        return reward;
    }

    return reward + gamma * maxExpectedUtility;
}
```

Рисунок 3.2.3 — Вычисление полезности состояния

3.3 Обновление Utility

Метод updateUtilities выполняет итеративное обновление значений полезности для всех станций сети. Процесс обновления повторяется несколько раз для обеспечения сходимости значений к стабильному состоянию, что соответствует теоретическим основам итерации по ценностям в марковских процессах принятия решений.

```
private void updateUtilities(String target) {  1 usage
    Map<String, Double> newUtilities = new HashMap<>();

    for (String station : graph.keySet()) {
        newUtilities.put(station, calculateUtility(station, target));
    }

    utilities.putAll(newUtilities);
}
```

Рисунок 3.3.1 — Обновление Utility для всех станций

На каждой итерации для каждой станции заново вычисляется значение полезности на основе текущих значений полезности соседних станций. Такой подход позволяет распространять информацию о вознаграждениях по всей сети, обеспечивая согласованность utility-функций.

Использование коэффициента дисконтирования gamma обеспечивает баланс между учетом немедленных и будущих вознаграждений. Значение gamma = 0,9 указывает на то, что агент в значительной степени учитывает будущие последствия своих действий при принятии решений.

4 Реализация графического интерфейса

4.1 Класс MapPanel

Класс MapPanel наследуется от JPanel и отвечает за визуализацию всей карты метро и связанных с ней элементов. Панель использует кастомную отрисовку для создания интуитивно понятного и информативного представления транспортной сети.

Система цветового кодирования реализована через словарь lineColors, который сопоставляет названия линий метро с соответствующими цветами. Красный цвет используется для Bakerloo Line, желтый - для Central Line, зеленый - для Piccadilly Line и синий - для Northern Line.

Метод paintComponent организует процесс отрисовки всех элементов карты в определенной последовательности: сначала соединения между станциями, затем подписи расстояний, сами станции, выделенный кратчайший путь, агент и вспомогательные элементы. Такой порядок обеспечивает корректное наложение visual элементов.

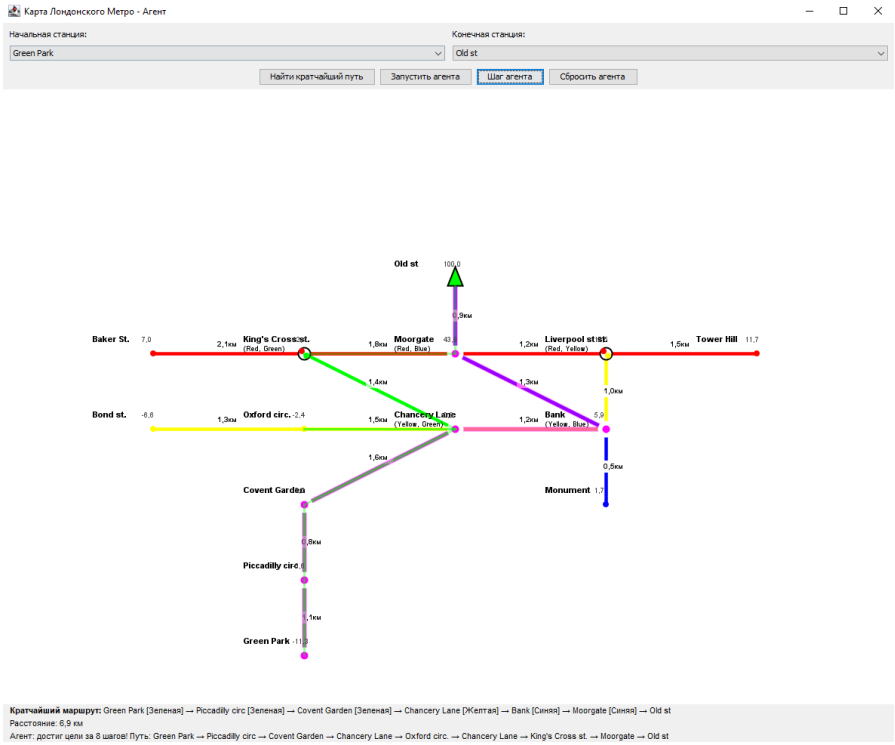

Рисунок 4.1.1 — Визуальное представление программы

4.2 Визуализация элементов карты

Метод drawConnections отрисовывает линии соединений между станциями с использованием толщины 4 пикселя и цветов, соответствующих линиям метро. Это создает четкое визуальное различение различных маршрутов на карте.

Метод drawStations реализует сложную систему отображения станций, учитывающую их тип и принадлежность к линиям. Станции пересадок отображаются как крупные круги с несколькими цветными точками внутри, представляющими различные линии. Обычные станции показываются как отдельные цветные точки.

Система автоматического позиционирования подписей в drawStationLabel обеспечивает читаемость текста независимо от расположения станции на карте. Для станций в верхней части карты подписи размещаются снизу, для станций в нижней части - сверху, а для станций по краям карты используется смещение для предотвращения выхода текста за границы.

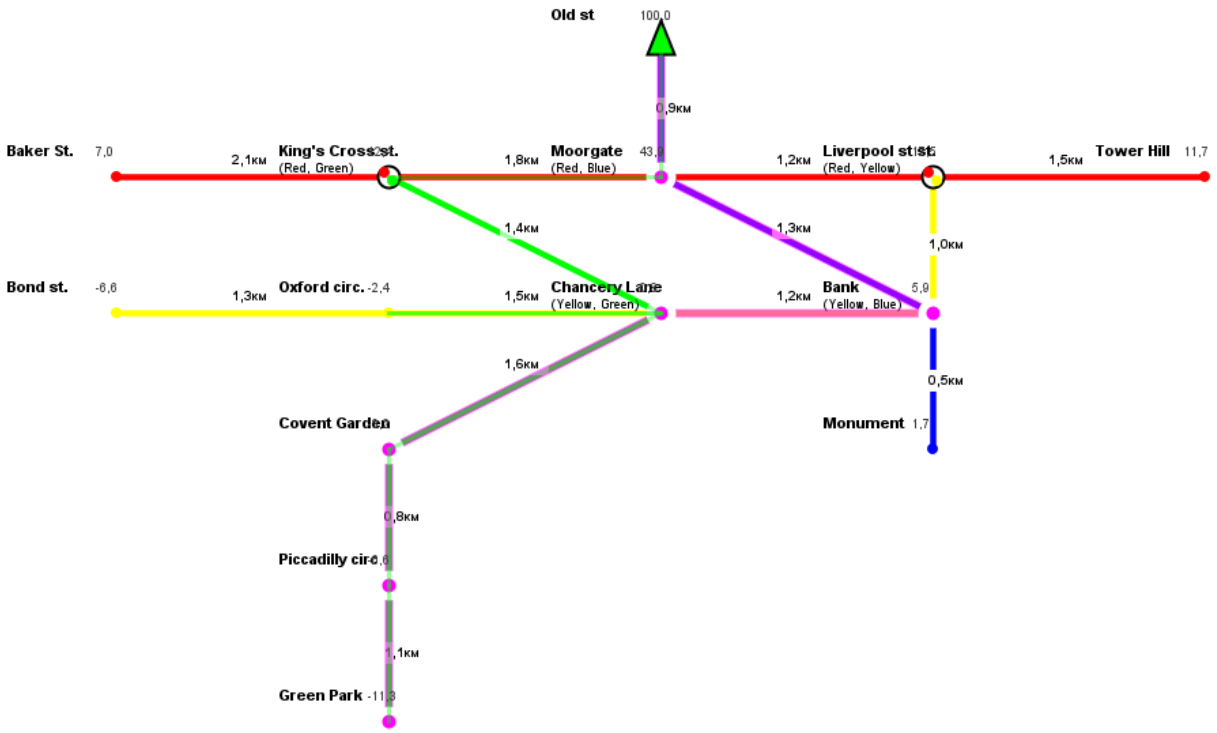

Рисунок 4.2.1 — Визуальное представление карты метро с подписями

4.3 Отображение агента и служебной информации

Метод drawAgent визуализирует текущее положение и путь интеллектуального агента. Агент представлен в виде зеленого треугольника, направленного острием вверх, что создает интуитивное представление о направлении движения. История перемещений агента отображается в виде полупрозрачной зеленой линии.

Рисунок 4.3.1 — Визуальное представление агента

Метод drawUtilities отображает текущие значения utility-функций для всех станций, что позволяет наблюдать процесс обучения агента в реальном времени. Значения отображаются в темно-сером цвете над соответствующими станциями с форматированием до одного десятичного знака.

Легенда карты, реализованная в методе drawLegend, предоставляет справочную информацию о цветах линий, обозначении агента и принципах отображения станций пересадок. Легенда размещается в правом верхнем углу карты на полупрозрачном фоне для обеспечения читаемости без перекрытия основных элементов.

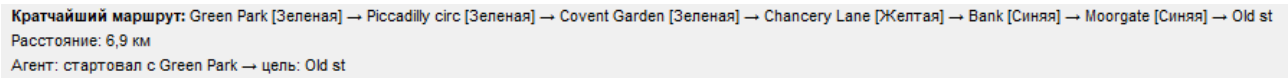

Рисунок 4.3.2 — Легенда

5 Система управления агентом

5.1 Инициализация и запуск агента

Метод startAgent выполняет подготовку агента к началу навигации. Процесс включает выбор начальной и конечной станций, проверку корректности введенных данных, инициализацию utility-функций и сброс состояния агента.

Перед началом движения агента выполняется предварительное обучение utility-функций через 10 итераций обновления. Это обеспечивает наличие начальных ориентиров для принятия решений агентом на первых шагах навигации.

Система отслеживания состояния агента инициализирует список agentPath для записи истории перемещений, счетчик agentSteps для подсчета выполненных действий и флаг agentReachedTarget для индикации достижения цели. Начальное положение агента устанавливается на выбранную станцию отправления.

5.2 Пошаговое выполнение и принятие решений

Метод stepAgent реализует логику одного шага движения агента по карте метро. На каждом шаге агент анализирует текущее положение и на основе utility-функций выбирает следующую станцию для перемещения.

Процесс выбора следующей станции в selectNextStationByUtility сочетает детерминированный подход на основе максимальной полезности со стохастическим элементом. С вероятностью 10% агент выбирает случайное направление, что имитирует элемент неопределенности в поведении и предотвращает зацикливание в локальных оптимумах.

После каждого перемещения обновляется состояние агента, увеличивается счетчик шагов и проверяется достижение целевой станции. Информация о текущем положении, количестве шагов и значении utility отображается в интерфейсе для наблюдения за процессом навигации.

5.3 Управление и сброс состояния

Метод resetAgent обеспечивает возможность полного сброса состояния агента без изменения конфигурации карты или выбранных станций. Это позволяет пользователю повторно запускать процесс навигации из тех же начальных условий для наблюдения различных сценариев поведения.

Система управления через графические элементы включает кнопки для запуска агента, пошагового выполнения и сброса состояния. Кнопка пошагового выполнения активируется только после инициализации агента и деактивируется после достижения цели, что предотвращает некорректные действия пользователя.

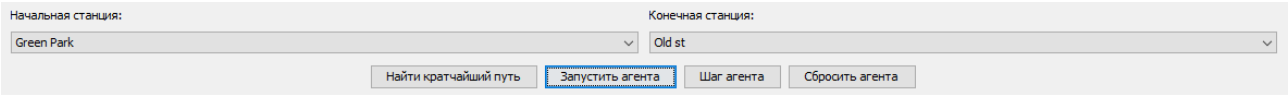

Рисунок 5.3.1 — Система управления

Интеграция с комбо-боксами выбора станций обеспечивает согласованность данных между различными компонентами интерфейса. Изменение выбранных станций автоматически triggers пересчет кратчайшего пути и обновление отображения на карте.

Приложение

Программный код на языке Java, моделирующий прогулку по лондонскому метро:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.util.*;
import java.util.List;

public class LondonTubeMap extends JFrame {
    private static final int WIDTH = 1200;
    private static final int HEIGHT = 800;
    private static final int MARGIN = 80;

    private Map<String, Station> stations = new HashMap<>();
    private Map<String, List<Connection>> graph = new HashMap<>();
    private String startStation = null;
    private String endStation = null;
    private List<String> shortestPath = new ArrayList<>();
    private double totalDistance = 0;

    // Агент и его состояние
    private String agentCurrentStation = null;
    private List<String> agentPath = new ArrayList<>();
    private int agentSteps = 0;
    private boolean agentReachedTarget = false;
    private Random random = new Random();

    // Параметры для вероятностной модели
    private double gamma = 0.9; // Коэффициент дисконтирования
    private Map<String, Double> utilities = new HashMap<>();

    private JComboBox<String> startComboBox, endComboBox;
    private JLabel pathLabel, distanceLabel, agentStatusLabel;
    private MapPanel mapPanel;
    private JButton startAgentButton, stepAgentButton, resetAgentButton;

    public LondonTubeMap() {
        setTitle("Карта Лондонского Метро - Агент");
        setSize(WIDTH, HEIGHT);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        initializeStations();
        initializeConnections();
        buildGraph();
        initializeUtilities();

        setupUI();
    }

    class Station {
        String name;
        int x, y;
        List<String> lines;
```

```
        Station(String name, int x, int y, List<String> lines) {
            this.name = name;
            this.x = x;
            this.y = y;
            this.lines = lines;
        }

        void addLine(String line) {
            if (!lines.contains(line)) {
                lines.add(line);
            }
        }
    }

    class Connection {
        String from, to;
        double distance;
        String line;

        Connection(String from, String to, double distance, String line) {
            this.from = from;
            this.to = to;
            this.distance = distance;
            this.line = line;
        }
    }

    private void initializeStations() {
        // Опускаем карту ниже - смещаем координаты Y вниз
        int centerX = WIDTH / 2;
        int centerY = HEIGHT / 2 + 100; // Смещаем центр ниже

        // Верхняя линия (красная)
        stations.put("Baker St.", new Station("Baker St.", centerX - 400, centerY - 150,
Arrays.asList("Red")));
        stations.put("King's Cross st.", new Station("King's Cross st.", centerX - 200,
centerY - 150, Arrays.asList("Red", "Green")));
        stations.put("Moorgate", new Station("Moorgate", centerX, centerY - 150,
Arrays.asList("Red", "Blue")));
        stations.put("Liverpool st st.", new Station("Liverpool st st.", centerX + 200,
centerY - 150, Arrays.asList("Red", "Yellow")));
        stations.put("Tower Hill", new Station("Tower Hill", centerX + 400, centerY -
150, Arrays.asList("Red")));

        // Средняя линия (желтая)
        stations.put("Bond st.", new Station("Bond st.", centerX - 400, centerY - 50,
Arrays.asList("Yellow")));
        stations.put("Oxford circ.", new Station("Oxford circ.", centerX - 200, centerY -
50, Arrays.asList("Yellow")));
        stations.put("Chancery Lane", new Station("Chancery Lane", centerX, centerY - 50,
Arrays.asList("Yellow", "Green")));
        stations.put("Bank", new Station("Bank", centerX + 200, centerY - 50,
Arrays.asList("Yellow", "Blue")));

        // Нижняя линия (зеленая)
        stations.put("Covent Garden", new Station("Covent Garden", centerX - 200, centerY
+ 50, Arrays.asList("Green")));
        stations.put("Piccadilly circ", new Station("Piccadilly circ", centerX - 200,
centerY + 150, Arrays.asList("Green")));
        stations.put("Green Park", new Station("Green Park", centerX - 200, centerY +
250, Arrays.asList("Green")));
```

```
        // Дополнительные станции (синяя)
        stations.put("Old st", new Station("Old st", centerX, centerY - 250,
Arrays.asList("Blue")));
        stations.put("Monument", new Station("Monument", centerX + 200, centerY + 50,
Arrays.asList("Blue")));
    }

    private void initializeConnections() {
        // Красная линия
        addConnection("Baker St.", "King's Cross st.", 2.1, "Red");
        addConnection("King's Cross st.", "Moorgate", 1.8, "Red");
        addConnection("Moorgate", "Liverpool st st.", 1.2, "Red");
        addConnection("Liverpool st st.", "Tower Hill", 1.5, "Red");

        // Желтая линия
        addConnection("Bond st.", "Oxford circ.", 1.3, "Yellow");
        addConnection("Oxford circ.", "Chancery Lane", 1.5, "Yellow");
        addConnection("Chancery Lane", "Bank", 1.2, "Yellow");
        addConnection("Bank", "Liverpool st st.", 1.0, "Yellow");

        // Зеленая линия
        addConnection("King's Cross st.", "Chancery Lane", 1.4, "Green");
        addConnection("Chancery Lane", "Covent Garden", 1.6, "Green");
        addConnection("Covent Garden", "Piccadilly circ", 0.8, "Green");
        addConnection("Piccadilly circ", "Green Park", 1.1, "Green");

        // Синяя линия
        addConnection("Old st", "Moorgate", 0.9, "Blue");
        addConnection("Moorgate", "Bank", 1.3, "Blue");
        addConnection("Bank", "Monument", 0.5, "Blue");

        // Пересадки между линиями
        addConnection("King's Cross st.", "King's Cross st.", 0.0, "Transfer");
        addConnection("Chancery Lane", "Chancery Lane", 0.0, "Transfer");
        addConnection("Bank", "Bank", 0.0, "Transfer");
        addConnection("Moorgate", "Moorgate", 0.0, "Transfer");
        addConnection("Liverpool st st.", "Liverpool st st.", 0.0, "Transfer");
    }

    private void addConnection(String from, String to, double distance, String line) {
        graph.putIfAbsent(from, new ArrayList<>());
        graph.putIfAbsent(to, new ArrayList<>());

        if (!from.equals(to) || line.equals("Transfer")) {
            graph.get(from).add(new Connection(from, to, distance, line));
            if (!from.equals(to)) {
                graph.get(to).add(new Connection(to, from, distance, line));
            }
        }
    }

    private void buildGraph() {
        // Граф уже построен в initializeConnections
    }

    private void initializeUtilities() {
        // Инициализация utility-функций для всех станций
        for (String station : graph.keySet()) {
            utilities.put(station, 0.0);
        }
```

```
}

private void setupUI() {
    setLayout(new BorderLayout());

    // Панель управления
    JPanel controlPanel = new JPanel(new GridLayout(2, 2, 10, 5));
    controlPanel.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 10));

    String[] stationNames = stations.keySet().toArray(new String[0]);
    Arrays.sort(stationNames);

    startComboBox = new JComboBox<>(stationNames);
    endComboBox = new JComboBox<>(stationNames);

    endComboBox.setSelectedItem("Tower Hill");

    JButton findPathButton = new JButton("Найти кратчайший путь");
    findPathButton.addActionListener(e -> findShortestPath());

    controlPanel.add(new JLabel("Начальная станция:"));
    controlPanel.add(new JLabel("Конечная станция:"));
    controlPanel.add(startComboBox);
    controlPanel.add(endComboBox);

    // Кнопки управления агентом
    startAgentButton = new JButton("Запустить агента");
    stepAgentButton = new JButton("Шаг агента");
    resetAgentButton = new JButton("Сбросить агента");

    startAgentButton.addActionListener(e -> startAgent());
    stepAgentButton.addActionListener(e -> stepAgent());
    resetAgentButton.addActionListener(e -> resetAgent());

    JPanel buttonPanel = new JPanel(new FlowLayout());
    buttonPanel.add(findPathButton);
    buttonPanel.add(startAgentButton);
    buttonPanel.add(stepAgentButton);
    buttonPanel.add(resetAgentButton);

    JPanel topPanel = new JPanel(new BorderLayout());
    topPanel.add(controlPanel, BorderLayout.CENTER);
    topPanel.add(buttonPanel, BorderLayout.SOUTH);

    // Панель результатов
    JPanel resultPanel = new JPanel(new GridLayout(3, 1));
    pathLabel = new JLabel("Выберите станции");
    distanceLabel = new JLabel("Расстояние: 0.0 км");
    agentStatusLabel = new JLabel("Агент: не активен");

    pathLabel.setBorder(BorderFactory.createEmptyBorder(2, 10, 2, 10));
    distanceLabel.setBorder(BorderFactory.createEmptyBorder(2, 10, 2, 10));
    agentStatusLabel.setBorder(BorderFactory.createEmptyBorder(2, 10, 2, 10));

    Font smallFont = new Font("Arial", Font.PLAIN, 11);
    pathLabel.setFont(smallFont);
    distanceLabel.setFont(smallFont);
    agentStatusLabel.setFont(smallFont);

    resultPanel.add(pathLabel);
    resultPanel.add(distanceLabel);
```

```
        resultPanel.add(agentStatusLabel);

        // Основная панель карты
        mapPanel = new MapPanel();

        mapPanel.addMouseListener(new MapMouseListener());

        add(topPanel, BorderLayout.NORTH);
        add(mapPanel, BorderLayout.CENTER);
        add(resultPanel, BorderLayout.SOUTH);
    }

    // Функция вознаграждения R(s)
    private double getReward(String station, String target) {
        if (station.equals(target)) {
            return 100.0; // Большое вознаграждение за достижение цели
        }

        // Отрицательное вознаграждение пропорционально расстоянию
        Station currentStation = stations.get(station);
        Station targetStation = stations.get(target);

        if (currentStation != null && targetStation != null) {
            double distance = Math.sqrt(
                    Math.pow(currentStation.x - targetStation.x, 2) +
                            Math.pow(currentStation.y - targetStation.y, 2)
            );
            return -distance / 100.0; // Небольшое отрицательное вознаграждение
        }

        return -1.0; // Стандартное отрицательное вознаграждение
    }

    // Функция перехода T(s, a, s') - вероятность перехода из s в s' при действии a
    private double getTransitionProbability(String from, String to, String target) {
        List<Connection> connections = graph.get(from);
        if (connections == null) return 0.0;

        // Базовые вероятности на основе расстояния
        double totalWeight = 0.0;
        Map<String, Double> weights = new HashMap<>();

        for (Connection conn : connections) {
            if (!conn.from.equals(conn.to)) { // Исключаем пересадки на себя
                double weight = 1.0 / (conn.distance + 0.1); // Чем ближе, тем вероятнее

                // Увеличиваем вес для направлений к цели
                if (isMovingTowardTarget(conn.to, from, target)) {
                    weight *= 2.0;
                }

                weights.put(conn.to, weight);
                totalWeight += weight;
            }
        }

        if (totalWeight == 0.0 || !weights.containsKey(to)) {
            return 0.0;
        }

        return weights.get(to) / totalWeight;
```

```
    }

    private boolean isMovingTowardTarget(String candidate, String current, String target)
{
        Station candidateStation = stations.get(candidate);
        Station currentStation = stations.get(current);
        Station targetStation = stations.get(target);

        if (candidateStation == null || currentStation == null || targetStation == null)
{
            return false;
        }

        double currentDistance = Math.sqrt(
                Math.pow(currentStation.x - targetStation.x, 2) +
                        Math.pow(currentStation.y - targetStation.y, 2)
        );

        double candidateDistance = Math.sqrt(
                Math.pow(candidateStation.x - targetStation.x, 2) +
                        Math.pow(candidateStation.y - targetStation.y, 2)
        );

        return candidateDistance < currentDistance;
    }

    // Вычисление utility по формуле: U(s) = R(s) + γ * max(Σ T(s,a,s') * U(s'))
    private double calculateUtility(String station, String target) {
        double reward = getReward(station, target);

        if (station.equals(target)) {
            return reward; // Целевое состояние
        }

        List<Connection> connections = graph.get(station);
        if (connections == null) return reward;

        double maxExpectedUtility = Double.NEGATIVE_INFINITY;

        // Для каждого возможного действия (перехода) вычисляем ожидаемую utility
        for (Connection conn : connections) {
            if (!conn.from.equals(conn.to)) {
                String nextStation = conn.to;
                double transitionProb = getTransitionProbability(station, nextStation,
target);
                double expectedUtility = transitionProb *
utilities.getOrDefault(nextStation, 0.0);

                if (expectedUtility > maxExpectedUtility) {
                    maxExpectedUtility = expectedUtility;
                }
            }
        }

        if (maxExpectedUtility == Double.NEGATIVE_INFINITY) {
            return reward;
        }

        return reward + gamma * maxExpectedUtility;
    }
```

```
    // Обновление utilities для всех станций
    private void updateUtilities(String target) {
        Map<String, Double> newUtilities = new HashMap<>();

        for (String station : graph.keySet()) {
            newUtilities.put(station, calculateUtility(station, target));
        }

        utilities.putAll(newUtilities);
    }

    // Стохастическое движение агента на основе utility-функций
    private void startAgent() {
        startStation = (String) startComboBox.getSelectedItem();
        endStation = (String) endComboBox.getSelectedItem();

        if (startStation == null || endStation == null) {
            JOptionPane.showMessageDialog(this, "Выберите начальную и конечную станции");
            return;
        }

        if (startStation.equals(endStation)) {
            JOptionPane.showMessageDialog(this, "Выберите разные станции");
            return;
        }

        // Обновляем utility-функции перед началом движения
        for (int i = 0; i < 10; i++) { // Несколько итераций для сходимости
            updateUtilities(endStation);
        }

        agentCurrentStation = startStation;
        agentPath.clear();
        agentPath.add(startStation);
        agentSteps = 0;
        agentReachedTarget = false;

        agentStatusLabel.setText("Агент: стартовал с " + startStation + " → цель: " +
endStation);
        mapPanel.repaint();
    }

    private void stepAgent() {
        if (agentCurrentStation == null || agentReachedTarget) {
            return;
        }

        agentSteps++;

        // Выбор следующей станции на основе utility-функций
        String nextStation = selectNextStationByUtility(agentCurrentStation);

        // Обновляем позицию агента
        agentCurrentStation = nextStation;
        agentPath.add(nextStation);

        // Проверяем достижение цели
        if (agentCurrentStation.equals(endStation)) {
            agentReachedTarget = true;
            agentStatusLabel.setText("Агент: достиг цели за " + agentSteps + " шагов!
Путь: " + String.join(" → ", agentPath));
```

```
        } else {
            agentStatusLabel.setText("Агент: шаг " + agentSteps + ", текущая позиция: " +
agentCurrentStation +
                    " → цель: " + endStation + " | Utility: " + String.format("%.2f",
utilities.get(agentCurrentStation)));
        }

        mapPanel.repaint();
    }

    private String selectNextStationByUtility(String currentStation) {
        List<Connection> connections = graph.get(currentStation);
        if (connections == null || connections.isEmpty()) {
            return currentStation;
        }

        // Выбираем станцию с максимальной utility
        String bestStation = currentStation;
        double maxUtility = Double.NEGATIVE_INFINITY;

        for (Connection conn : connections) {
            if (!conn.from.equals(conn.to)) {
                String nextStation = conn.to;
                double utility = utilities.getOrDefault(nextStation, 0.0);

                if (utility > maxUtility) {
                    maxUtility = utility;
                    bestStation = nextStation;
                }
            }
        }

        // Добавляем стохастичность: с небольшой вероятностью выбираем случайное
направление
        if (random.nextDouble() < 0.1) { // 10% вероятность случайного выбора
            List<String> possibleStations = new ArrayList<>();
            for (Connection conn : connections) {
                if (!conn.from.equals(conn.to)) {
                    possibleStations.add(conn.to);
                }
            }
            if (!possibleStations.isEmpty()) {
                return possibleStations.get(random.nextInt(possibleStations.size()));
            }
        }

        return bestStation;
    }

    private void resetAgent() {
        agentCurrentStation = null;
        agentPath.clear();
        agentSteps = 0;
        agentReachedTarget = false;
        agentStatusLabel.setText("Агент: сброшен");
        mapPanel.repaint();
    }

    private void findShortestPath() {
        startStation = (String) startComboBox.getSelectedItem();
        endStation = (String) endComboBox.getSelectedItem();
```

```
        if (startStation == null || endStation == null) {
            JOptionPane.showMessageDialog(this, "Выберите начальную и конечную станции");
            return;
        }

        if (startStation.equals(endStation)) {
            JOptionPane.showMessageDialog(this, "Выберите разные станции");
            return;
        }

        shortestPath = dijkstra(startStation, endStation);
        mapPanel.repaint();

        if (shortestPath.isEmpty()) {
            pathLabel.setText("Путь не найден");
            distanceLabel.setText("Расстояние: 0.0 км");
        } else {
            updatePathInfo();
        }
    }

    private List<String> dijkstra(String start, String end) {
        Map<String, Double> distances = new HashMap<>();
        Map<String, String> previous = new HashMap<>();
        PriorityQueue<String> queue = new PriorityQueue<>(
                Comparator.comparingDouble(station -> distances.getOrDefault(station,
Double.MAX_VALUE))
        );

        for (String station : graph.keySet()) {
            distances.put(station, Double.MAX_VALUE);
        }
        distances.put(start, 0.0);
        queue.add(start);

        while (!queue.isEmpty()) {
            String current = queue.poll();

            if (current.equals(end)) {
                break;
            }

            for (Connection conn : graph.get(current)) {
                double newDist = distances.get(current) + conn.distance;
                if (newDist < distances.get(conn.to)) {
                    distances.put(conn.to, newDist);
                    previous.put(conn.to, current);
                    queue.add(conn.to);
                }
            }
        }

        List<String> path = new ArrayList<>();
        String current = end;
        totalDistance = distances.get(end);

        while (current != null) {
            path.add(0, current);
            current = previous.get(current);
        }
```

```
        return path.get(0).equals(start) ? path : new ArrayList<>();
    }

    private void updatePathInfo() {
        StringBuilder pathText = new StringBuilder("<html><b>Кратчайший маршрут:</b> ");
        for (int i = 0; i < shortestPath.size(); i++) {
            if (i > 0) {
                String from = shortestPath.get(i-1);
                String to = shortestPath.get(i);
                String lineInfo = getLineInfoForConnection(from, to);
                pathText.append(" ").append(lineInfo).append(" → ");
            }
            pathText.append(shortestPath.get(i));
        }
        pathText.append("</html>");

        pathLabel.setText(pathText.toString());
        distanceLabel.setText(String.format("Расстояние: %.1f км", totalDistance));
    }

    private String getLineInfoForConnection(String from, String to) {
        List<Connection> connections = graph.get(from);
        if (connections != null) {
            for (Connection conn : connections) {
                if (conn.to.equals(to) && !conn.line.equals("Transfer")) {
                    switch (conn.line) {
                        case "Red": return "[Красная]";
                        case "Yellow": return "[Желтая]";
                        case "Green": return "[Зеленая]";
                        case "Blue": return "[Синяя]";
                    }
                }
            }
        }
        return "[Пересадка]";
    }

    class MapPanel extends JPanel {
        private final Map<String, Color> lineColors = Map.of(
                "Red", Color.RED,
                "Yellow", Color.YELLOW,
                "Green", Color.GREEN,
                "Blue", Color.BLUE,
                "Transfer", Color.GRAY
        );

        public MapPanel() {
            setPreferredSize(new Dimension(WIDTH, HEIGHT));
            setBackground(Color.WHITE);
        }

        @Override
        protected void paintComponent(Graphics g) {
            super.paintComponent(g);
            Graphics2D g2d = (Graphics2D) g;
            g2d.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
RenderingHints.VALUE_ANTIALIAS_ON);

            drawConnections(g2d);
            drawDistanceLabels(g2d);
```

```
            drawStations(g2d);
            drawShortestPath(g2d);
            drawAgent(g2d);
            drawLegend(g2d);
            drawUtilities(g2d);
        }

        private void drawConnections(Graphics2D g2d) {
            for (List<Connection> connections : graph.values()) {
                for (Connection conn : connections) {
                    if (!conn.from.equals(conn.to)) {
                        Station fromStation = stations.get(conn.from);
                        Station toStation = stations.get(conn.to);

                        if (fromStation != null && toStation != null) {
                            g2d.setColor(lineColors.get(conn.line));
                            g2d.setStroke(new BasicStroke(4));
                            g2d.drawLine(fromStation.x, fromStation.y, toStation.x,
toStation.y);
                        }
                    }
                }
            }
        }

        private void drawDistanceLabels(Graphics2D g2d) {
            g2d.setFont(new Font("Arial", Font.BOLD, 10));

            for (List<Connection> connections : graph.values()) {
                for (Connection conn : connections) {
                    if (!conn.from.equals(conn.to) && conn.distance > 0) {
                        Station fromStation = stations.get(conn.from);
                        Station toStation = stations.get(conn.to);

                        if (fromStation != null && toStation != null) {
                            drawDistanceLabel(g2d, fromStation, toStation,
conn.distance);
                        }
                    }
                }
            }
        }

        private void drawDistanceLabel(Graphics2D g2d, Station from, Station to, double
distance) {
            int midX = (from.x + to.x) / 2;
            int midY = (from.y + to.y) / 2;

            int offsetX = 0, offsetY = 0;
            if (Math.abs(from.x - to.x) > Math.abs(from.y - to.y)) {
                offsetY = -12;
            } else {
                offsetX = 12;
            }

            g2d.setColor(new Color(255, 255, 255, 220));
            g2d.fillRoundRect(midX + offsetX - 18, midY + offsetY - 8, 36, 16, 8, 8);

            g2d.setColor(Color.BLACK);
            String distanceText = String.format("%.1fкм", distance);
            g2d.drawString(distanceText, midX + offsetX - 15, midY + offsetY + 3);
```

```
        }

        private void drawStations(Graphics2D g2d) {
            g2d.setFont(new Font("Arial", Font.BOLD, 11));

            for (Station station : stations.values()) {
                drawStationCircle(g2d, station);
                g2d.setColor(Color.BLACK);
                drawStationLabel(g2d, station);
            }
        }

        private void drawStationCircle(Graphics2D g2d, Station station) {
            int size = 12;
            if (station.lines.size() > 1) {
                size = 16;
            }

            if (station.lines.size() > 1) {
                g2d.setColor(Color.WHITE);
                g2d.fillOval(station.x - size/2, station.y - size/2, size, size);
                g2d.setColor(Color.BLACK);
                g2d.setStroke(new BasicStroke(2));
                g2d.drawOval(station.x - size/2, station.y - size/2, size, size);
            }

            int smallSize = 8;
            int offset = 0;
            for (String line : station.lines) {
                g2d.setColor(lineColors.get(line));
                int xOffset = 0, yOffset = 0;

                if (station.lines.size() == 2) {
                    if (offset == 0) {
                        xOffset = -3; yOffset = -3;
                    } else {
                        xOffset = 3; yOffset = 3;
                    }
                } else if (station.lines.size() > 2) {
                    xOffset = (offset % 2 == 0) ? -2 : 2;
                    yOffset = (offset / 2 == 0) ? -2 : 2;
                }

                g2d.fillOval(station.x - smallSize/2 + xOffset, station.y - smallSize/2 +
yOffset, smallSize, smallSize);
                offset++;
            }
        }

        private void drawStationLabel(Graphics2D g2d, Station station) {
            int offsetX = 10;
            int offsetY = -10;

            // Автоматическое позиционирование подписей
            if (station.y < HEIGHT / 4) {
                offsetY = 15;
            } else if (station.y > 3 * HEIGHT / 4) {
                offsetY = -15;
            }

            if (station.x > 3 * WIDTH / 4) {
```

```
                offsetX = -80;
            }

            g2d.drawString(station.name, station.x + offsetX, station.y + offsetY);

            if (station.lines.size() > 1) {
                g2d.setFont(new Font("Arial", Font.PLAIN, 9));
                String linesText = String.join(", ", station.lines);
                g2d.drawString("(" + linesText + ")", station.x + offsetX, station.y +
offsetY + 12);
                g2d.setFont(new Font("Arial", Font.BOLD, 11));
            }
        }

        private void drawShortestPath(Graphics2D g2d) {
            if (shortestPath.size() < 2) return;

            g2d.setColor(new Color(255, 0, 255, 150));
            g2d.setStroke(new BasicStroke(6));

            for (int i = 0; i < shortestPath.size() - 1; i++) {
                Station from = stations.get(shortestPath.get(i));
                Station to = stations.get(shortestPath.get(i + 1));

                if (from != null && to != null) {
                    g2d.drawLine(from.x, from.y, to.x, to.y);
                }
            }

            for (String stationName : shortestPath) {
                Station station = stations.get(stationName);
                if (station != null) {
                    g2d.setColor(new Color(255, 0, 255));
                    g2d.fillOval(station.x - 8, station.y - 8, 16, 16);
                    g2d.setColor(Color.WHITE);
                    g2d.drawOval(station.x - 8, station.y - 8, 16, 16);
                }
            }
        }

        private void drawAgent(Graphics2D g2d) {
            if (agentCurrentStation != null) {
                Station agentStation = stations.get(agentCurrentStation);
                if (agentStation != null) {
                    // Рисуем агента как зеленый треугольник
                    g2d.setColor(Color.GREEN);
                    int[] xPoints = {agentStation.x, agentStation.x - 10, agentStation.x
+ 10};
                    int[] yPoints = {agentStation.y - 15, agentStation.y + 10,
agentStation.y + 10};
                    g2d.fillPolygon(xPoints, yPoints, 3);

                    // Обводка агента
                    g2d.setColor(Color.BLACK);
                    g2d.setStroke(new BasicStroke(2));
                    g2d.drawPolygon(xPoints, yPoints, 3);

                    // Путь агента
                    if (agentPath.size() > 1) {
                        g2d.setColor(new Color(0, 255, 0, 100));
                        g2d.setStroke(new BasicStroke(3));
```

```
                for (int i = 0; i < agentPath.size() - 1; i++) {
                    Station from = stations.get(agentPath.get(i));
                    Station to = stations.get(agentPath.get(i + 1));

                    if (from != null && to != null) {
                        g2d.drawLine(from.x, from.y, to.x, to.y);
                    }
                }
            }
        }
    }
}

private void drawUtilities(Graphics2D g2d) {
    if (agentCurrentStation == null) return;

    g2d.setFont(new Font("Arial", Font.PLAIN, 9));
    g2d.setColor(Color.DARK_GRAY);

    for (Station station : stations.values()) {
        double utility = utilities.getOrDefault(station.name, 0.0);
        String utilityText = String.format("%.1f", utility);
        g2d.drawString(utilityText, station.x - 15, station.y - 15);
    }
}

private void drawLegend(Graphics2D g2d) {
    int legendX = WIDTH - 200;
    int legendY = 50;

    g2d.setColor(new Color(255, 255, 255, 220));
    g2d.fillRoundRect(legendX - 10, legendY - 10, 180, 200, 10, 10);

    g2d.setColor(Color.BLACK);
    g2d.setFont(new Font("Arial", Font.BOLD, 12));
    g2d.drawString("Легенда:", legendX, legendY + 10);

    String[] lines = {"Красная: Bakerloo Line", "Желтая: Central Line",
            "Зеленая: Piccadilly Line", "Синяя: Northern Line"};
    Color[] colors = {Color.RED, Color.YELLOW, Color.GREEN, Color.BLUE};

    for (int i = 0; i < lines.length; i++) {
        g2d.setColor(colors[i]);
        g2d.fillRect(legendX, legendY + 25 + i * 25, 15, 3);
        g2d.setColor(Color.BLACK);
        g2d.drawString(lines[i], legendX + 20, legendY + 30 + i * 25);
    }

    // Агент
    g2d.setColor(Color.GREEN);
    int[] xPoints = {legendX + 7, legendX, legendX + 14};
    int[] yPoints = {legendY + 125, legendY + 140, legendY + 140};
    g2d.fillPolygon(xPoints, yPoints, 3);
    g2d.setColor(Color.BLACK);
    g2d.drawString("Агент", legendX + 20, legendY + 135);

    // Utility
    g2d.setColor(Color.DARK_GRAY);
    g2d.drawString("Utility значения", legendX, legendY + 160);
    g2d.drawString("показаны сверху", legendX, legendY + 175);
```

```
            g2d.setColor(Color.BLACK);
            g2d.drawString("Станции пересадок:", legendX, legendY + 195);
            g2d.drawString("• Несколько кругов", legendX + 10, legendY + 210);
        }
    }

    class MapMouseListener extends MouseAdapter {
        @Override
        public void mouseClicked(MouseEvent e) {
            for (Station station : stations.values()) {
                double distance = Math.sqrt(Math.pow(e.getX() - station.x, 2) +
Math.pow(e.getY() - station.y, 2));
                if (distance < 15) {
                    if (SwingUtilities.isLeftMouseButton(e)) {
                        startComboBox.setSelectedItem(station.name);
                    } else if (SwingUtilities.isRightMouseButton(e)) {
                        endComboBox.setSelectedItem(station.name);
                    }
                    findShortestPath();
                    break;
                }
            }
        }
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            try {
                UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());
            } catch (Exception e) {
                e.printStackTrace();
            }

            LondonTubeMap map = new LondonTubeMap();
            map.setVisible(true);
        });
    }
}
```